



Understanding Partitions

A Dive Inside the Hard Disk

This article, which is part of the series on Linux device drivers, takes you on a tour inside a hard disk.

Part—14

“Doesn't it sound like a mechanical engineering subject: The design of the hard disk?” questioned Shweta. “Yes, it does. But understanding it gives us an insight into its programming aspect,” reasoned Pugs, while waiting for the commencement of the seminar on storage systems.

The seminar started with a few hard disks in the presenter's hand and then a dive into her system, showing the output of `fdisk -l` (Figure 1). The first line shows the hard disk size in human-friendly format and in bytes. The second line mentions the number of logical heads, logical sectors per track, and the actual number of cylinders on the disk—together known as the *geometry* of the disk. The 255 heads indicate the number of platters or disks, as one read-write head is needed per disk. Let's number them, say D1, D2, ... D255. Now, each disk would have the same number of concentric circular tracks, starting from the outside to the inside. In the above case, there are 60801 such tracks per disk. Let's number them, say T1, T2, ... T60801. And a particular track number from all the disks forms a cylinder of the same number. For example, tracks T2 from D1, D2, ... D255 will together form the cylinder C2. Now, each track has the same

number of logical sectors—63 in our case, say S1, S2, ... S63. And each sector is typically 512 bytes. Given this data, one can actually compute the total usable hard disk size, using the following formula:

Usable hard disk size in bytes = (Number of heads or disks)
* (Number of tracks per disk) * (Number of sectors per
track) * (Number of bytes per sector, i.e. sector size).

For the disk under consideration, it would be: $255 * 60801 * 63 * 512$ bytes = 500105249280 bytes. Note that this number may be slightly less than the actual hard disk (500107862016 bytes, in our case). The reason is that the formula doesn't consider the bytes in the last partial or incomplete cylinder. The primary reason for that is the difference between today's technology of organising the actual physical disk geometry and the traditional geometry representation using heads, cylinders and sectors. Note that in the `fdisk` output, we referred to the heads and sectors per track as *logical* not physical. One may ask that if today's disks don't have such physical geometry concepts, then why still maintain them and represent them in a logical form? The main reason is to be able to continue with the same concepts of